Admin Dashboard — QA / Technical Audit

A finding-by-finding response to the admin QA audit: the evidence for what was already implemented before the audit was written, the three defects it did not find that we did, and the attached endpoint-by-role matrix it asked for by name.

Aajoo Homes — Admin Dashboard · QA / Technical Audit
Prepared 6 September 2026 by the Zyphex Tech development team
Verified against the live development backend and database

1. Response summary

This answers **AAJOO HOMES ADMIN DASHBOARD — FULL QA / TECHNICAL AUDIT**, ID by ID, in the order the audit raises them.

One thing first, because it changes how the rest should be read. The audit states its own limitation honestly — it was performed against a supplied backend **archive**, `aajaoBackend-main (3)(1).zip`. That archive predates a large amount of work. **Most of the P0 and P1 findings were already implemented before the audit arrived**, several of them citing these exact IDs in the code comments. So the bulk of this response is not "we fixed it" — it is the evidence that it is already true, which is what the audit asked for and did not have access to.

Three things in it were *not* true, and are now.

The audit asks by name for an **endpoint × role authorization matrix**. It is attached as a separate document, generated from the route definitions rather than typed: 191 admin endpoints, every cell decided by calling the same functions the middleware calls.

Response at a glance	
Findings answered	**6 P0 + 6 P1 + 8 production + 5 database + 10 sections**
Already implemented, verified against the current code and the live system	**22**
Fixed during this review	**3**
Needs a setting only you can make	**2**
Called out rather than quietly changed	**1**
Backend	`2ba4345`
Admin frontend	`55c4f7d`
Migrations applied to the live database today	**4**
Backend test suite	**63/63 files pass**
Attached	`AAJOO_ADMIN_RBAC_MATRIX_2026-09-06` — 191 endpoints × 5 roles

2. Section 2 — Release-blocking P0

ID	Finding	Status	Evidence
ADM-P0-01	Finance mutation endpoints lack RBAC gates	**Already done**	Measured from the routes, not asserted: **27 of 27** finance endpoints carry `adminAuth` **and** an explicit `requireRole`. Nineteen are `FINANCE_READ` (super admin, finance, admin); **eight are `FINANCE_WRITE` — super admin and finance only** — and those eight are exactly the ones that move money: payout initiate, approve, reject, schedule create and update, invoice void, reconciliation resolve and run. An operations administrator cannot approve a payout. Covered by `tests/adminRbac.test.js`.
ADM-P0-02	Bundled Razorpay TEST credentials in source	**Already done**	Both keys come from the environment; there is no fallback literal. `usableForPayments` is false when the keys are absent, and false for a test key on a production deploy unless `ALLOW_TEST_PAYMENTS=true` is set deliberately — a separate, greppable variable rather than "leave NODE_ENV unset", because that would loosen the CORS allowlist and every other production check at the same time. It is announced at boot on every start.
ADM-P0-03	CORS allows all origins	**Already done**	HTTP and Socket.IO both use one allowlist function, not `origin: true` and not `'*'`. The socket carries negotiation messages and admin events, so it takes the same list.
ADM-P0-04	RBAC storage is incomplete	**Already done**	`tbl_admins.admin_role` exists, with a migration that backfilled it, and `config/adminRoles.js` is the single definition used by the migration, the model, the token, the middleware and the matrix test. Five roles: super admin, admin, finance, support, SEO manager. Exactly one of them — super admin — passes a gate it was not named in; `requireRole` used to treat *admin* that way, which made every gate in the system admit every administrator.
ADM-P0-05	Persistent admin mutation audit is incomplete	**Already done**	`tbl_admin_audit` is a durable ledger — actor, role, action, entity, entity id, before, after, reason, timestamp — written by approvals, KYC decisions, payouts, invoice voids, reconciliation, finance mutations, listing decisions, coupons and role changes. The login log still exists and is still only a login log; it is no longer the whole story. Four indexes on it.
ADM-P0-06	DB failure is not a hard readiness gate	**Already done**	Liveness and readiness are separated. The process does not exit on a transient database blip — killing a service over a reconnectable error is the worse failure — but the instance is marked **not ready**, and `/health/env` answers 503 while it is, so the platform routes around it.

3. Section 3 — High-severity security / authorization

ID	Finding	Status	Evidence
ADM-P1-01	Admin rate limiter uses the wrong claim	**Already done**	The bucket key reads `userId ?? id ?? adminId` from the verified claims and, failing that, decodes the token's payload without verifying it — deliberately, since a forged token can only move the forger into a different rate-limit bucket. Before this every signed-in administrator fell through to the IP bucket and shared one budget with everyone behind the same office connection, which is why admin screens started failing mid-session with nothing actually wrong.

ID	Finding	Status	Evidence
ADM-P1-02	Role changes rely on legacy `isAdmin`	**Already done**	The role column decides; `admin_isAdmin` is kept in sync as a legacy mirror and is never the authority. A token minted before a demotion does not keep its powers — the role is re-read from the row on every admin request, so deactivating or demoting an administrator takes effect immediately rather than when their token expires.
ADM-P1-03	Admin creation route uses `optionalAdminAuth`	**Already done, and deliberately**	The route is open *only* while `tbl_admins` is empty — otherwise the first administrator could never be created. From the second account onward the controller requires an authenticated **super admin** and refuses anything else, and a role that is not on the canonical list is rejected. Verified live: an unauthenticated `POST /admin/create` is answered with validation errors, and a valid body from an unauthenticated caller is refused.
ADM-P1-04	Finance reads lack explicit least-privilege gates	**Already done, and extended this review**	All 27 finance routes are gated (above). The audit's wider point — least privilege — was only half true: of **191 admin endpoints, 29 carry an explicit role gate**. The other 162 were written when every administrator was equivalent. `seo_manager` was already confined centrally; `support` **was not**, so a support login could approve a property, cancel a booking, delete a user or issue a coupon. Fixed — see section 15.
ADM-P1-05	KYC / private identity data needs stronger authorization	**Already done**	Identity, ownership and KYC documents are Cloudinary *authenticated* assets, reachable only through short-lived signed URLs — not by guessing a path. `uploads/` is no longer served as static files; invoices go through a signed route, because an invoice names the guest, the property, the dates and the amount. A support account can now read a host record and cannot reach the KYC decision endpoints.
ADM-P1-06	Object-level authorization must be tested everywhere	**Done, with one honest limit**	Every guest query carries the caller's id, every host query `property_host_id`, every payout the host id, every ticket both the user id and the ticket's role. A wrong id is answered *not found* rather than *forbidden*, so an error cannot be used to enumerate what exists. What we cannot do is drive the authenticated admin UI with real logins — see section 17.

4. Section 4 — Admin functional coverage

Every module the audit lists exists and is wired to the backend. Rather than repeat sixteen rows of "yes", here is what is worth knowing about each.

Module	State
Dashboard	KPIs, date filters, queues and recent activity. The four headline counts were reconciled with their own list screens — see section 9.
Users	Search, detail, verification, activation, deletion policy, bookings, support context, audit.
Hosts	Onboarding, KYC, verification, activation, properties, earnings, payouts, documents.
Properties	Search, detail, a real verification state machine (draft → submitted → approved/rejected/changes/suspended) with the legal transitions enforced, media, categories, availability, booking history.

Module	State
Bookings	Search, detail, status, amendment, cancellation and refund state, payment state, invoice, audit. An admin can no longer mark a booking paid when no payment was recorded.
Negotiations	Guest/host, property, full offer history, state, final price, timestamps, expiry, escalation, participant isolation.
Finance	Ledger, payouts, invoices, reconciliation, commission, revenue, tax, cashflow, exports, approval and rejection with reasons, audit.
Support	Tickets, threads, reply, status, closure — and, since today, guest tickets in the same queue as host tickets, labelled by who raised them.
Disputes	Queue, evidence, decision notes, linked booking, immutable history.
Referrals, Coupons, CMS, Reviews, Analytics	All present and backed by real endpoints.
Team / Roles	Creation, activation, role assignment, **session revocation**, audit.
System health	Database, payments, email, push, environment readiness — public answers ready/not-ready, detail behind a token.

5. Section 5 — Endpoint regression matrix

Attached in full as `AAJOO_ADMIN_RBAC_MATRIX_2026-09-06`: 191 endpoints across 29 route groups, with the expected outcome for each of the five roles. Generated from the route definitions and evaluated through the same functions the middleware uses, so it cannot drift from the server. Summarised:

Admin endpoints	**191**
Behind admin authentication	**189**
Deliberately unauthenticated	**2** — `POST /admin/login`, and `POST /admin/create`, which is open only while no admin exists and demands a super admin thereafter
With an explicit `requireRole`	**29**
Confined centrally instead	`seo_manager` and `support`, by named scope, default deny
Finance write endpoints reachable by an operations admin	**0**
Endpoints a support account may now change	**the 4 queues it owns**, out of 191

6. Section 6 — Database and data integrity

Requirement	Result
Migrations run from empty and against the current schema; order and rollback	Every migration is written idempotently — it checks whether the column, index or table is already there — so it runs on an empty database and on this one. Four were applied to the live database today, each with a `down`.
Foreign keys and orphan prevention	Present on the modular listing tables; the legacy tables use application-level scoping. Nothing in the admin or public queries returns an orphan, because every one filters by the owning id.
Indexes for booking date/status, payment ids, payout ids, KYC status, negotiation, tickets, audit	Bookings 10, payments 6, properties 8, support tickets 7, admin audit 4, users 5. **Three tables carried nothing but a primary key and now have indexes — see section 15.**
Monetary calculations use DECIMAL/NUMERIC	**This was not true. Sixteen money columns were `DOUBLE`. Fixed today — section 15.**
Financial mutations transactional and idempotent	Payment verification takes a row lock scoped to the caller; booking creation serialises per property; cancellation, settlement and payout each run in one transaction.
Soft-deleted users and properties never reappear	One visibility rule — `is_active = 1 AND is_deleted <> 1` — applied by search, the property endpoint, the sitemap and the public page, which answers a real 404.
Dashboard KPI counts use the same populations as their list screens	Reconciled — section 9.
Timezone handling	Dates are day-stamps in IST, computed by one helper shared by the server, the website and the app, so the three cannot disagree at a month boundary or across devices.
Concurrent admin updates cannot silently overwrite a newer decision	The listing state machine refuses an illegal transition rather than applying it; finance mutations are locked.
Audit before/after matches the committed state	The ledger is written inside the same transaction as the change.

7. Section 7 — Codebase-specific database findings

ID	Finding	Status
DB-01	No canonical finance/support role storage	**Already done** — `admin_role`, migrated and backfilled, with one definition file.
DB-02	Login logs are not a mutation audit	**Already done** — `tbl_admin_audit` is the mutation ledger; the login log stayed what it is.
DB-03	`tbl_admin_flags` mixes KYC and payout variance with disputes	**Answered** — investigated and settled: what the admin panel called "disputes" *were* KYC holds, all of them. They are separated now: compliance flags are one thing, customer disputes another, and the queue says which.

ID	Finding	Status
DB-04	Verify soft-delete filtering everywhere	**Done** — one rule, applied by every query listed above.
DB-05	Ledger, wallet, payout, invoice and reconciliation reconcile without unexplained deltas	**Done, and the arithmetic is now exact** — this is precisely what the DOUBLE columns threatened. See section 15.

---

8. Section 8 — Dashboard KPI

The audit's first three bullets describe a defect that existed and was fixed: the dashboard tiles and the list screens they link to were counting **different populations**.

- Users — the tile also required `isUser = 1` and `isActive = 1`; the user list filters on `isDelete` alone.
- Hosts — the tile also required `isActive = 1`; the host list does not.
- Properties — the tile also required `isVerify = 1` **and** `isActive = 1`, so it reported a smaller number than the screen it opened.

They now count the same populations, with the reason written beside them so the next person does not "tidy" one of them back. Revenue excludes cancelled and unpaid rows by an explicit status list rather than by assumption; empty data renders zero rather than null; pagination totals come from the same query as the rows.

---

9. Section 9 — Finance deep QA

Check	Result
Cannot approve a payout for another host by changing the id	Scoped by host id from the token, never from the body.
Finance role can perform only permitted finance actions	27 gated routes, 8 write-gated. Support, host and guest are all refused.
Payout approval requires valid state and eligibility	State checked; the host's identity check must be current.
Rejecting a payout requires a reason	Required.
Invoice void requires a reason and immutable audit	Both.
Reconciliation resolution requires notes and an actor	Both, in the ledger.
Duplicate payout approval is idempotent	Yes — and, fixed earlier today, a payout **request** can no longer exceed the available balance or be raised twice while one is pending. That check was a comment with nothing under it.

Check	Result
Duplicate payment/refund/webhook events do not duplicate ledger entries	The verification handler row-locks the payment; a replay returns success with no side effects.
Gross, commission, tax, net host earning and platform earning reconcile	Now with exact arithmetic — section 15.
Exports match the filtered screen	Yes, and are role-gated with the reads.

10. Section 10 — Negotiation control plane

Check	Result
Full thread with correct guest/host identity after alternating messages	Identity comes from **property ownership**, not from who sent last.
Final negotiated amount equals the booking amount	The agreed price is a server-issued coupon pinned to the offer's dates; the client cannot name a price or move the dates.
Expired offers cannot appear active	Expiry is evaluated server-side on every read and action.
Internal thresholds not exposed	`property_mini_price` and `property_ideal_price` are excluded for anyone but the owner — verified against the live public payload, both absent.
Every transition has timestamp and actor	Recorded in the offer ledger.
Admin cannot silently alter a negotiation	Admin mutations write to `tbl_admin_audit`.
Concurrent accept/counter resolve deterministically	One locking read holds the thread until the offer row is written. Before it, two requests a millisecond apart both saw no pending offer.

11. Section 11 — Support and disputes

Check	Result
Unique ID and complete thread	Yes; the guest side now gets a quotable reference (`AJ-000123`).
Status transitions validated	OPEN / PENDING / RESOLVED / CLOSED, and a reply into a closed ticket is refused rather than swallowed.
Closure and dispute resolution require a reason	Yes.
Ticket links to user/host/booking/property	Yes.
Compliance flags distinguishable from customer disputes	Yes — see DB-03.
Support roles cannot access financial mutation	**Yes — and as of this review they cannot access non-financial mutation either.** Section 15.
Sensitive PII masked by role	Bank details are masked; identity documents need a signed URL.

12. Section 12 — Production readiness

ID	Finding	Status
PROD-01	Payment fallback	**Done** — environment only, fails closed, `ALLOW_TEST_PAYMENTS` is the one explicit escape hatch and must be removed at go-live.
PROD-02	CORS	**Done** — one allowlist for HTTP and WebSocket.
PROD-03	DB failure	**Done** — readiness separated from liveness.
PROD-04	`/health/env` detail	**Done** — public callers get `{ready}` and nothing else; the inventory of which secrets are set requires `HEALTH_TOKEN`. Same treatment applied to `/health/push` today, which had been publishing the Firebase project id and service-account email to anyone.
PROD-05	`/db-test`	**Done** — answers a bare ok/unavailable with no driver text. Kept rather than deleted because the platform's health-check path is configured outside this repository and a 404 there would fail every deploy.
PROD-06	Uploads	**Done** — `uploads/` is not static; private documents are authenticated assets behind signed URLs.
PROD-07	Socket.IO origins and authentication	**Done** — origins allowlisted, and as of today a socket that cannot be identified is **refused**. It used to be admitted, and the anti-spoofing guard was skipped for exactly those sockets.
PROD-08	Admin limiter uses `adminId`	**Done.**

13. Section 13 — Admin frontend final-test requirements

Requirement	State
Dashboard cards link to a screen with the same population	Yes — section 8.
Loading, empty, error, retry and pagination states	Present on the admin tables.
Destructive actions confirm and handle the committed result	Yes.
Financial mutations show status and reference	Yes.
RBAC enforced by the backend, not by hiding UI	Yes — the attached matrix is the server's behaviour, not the menu's.
Refresh on a protected page revalidates	Yes, and a signed-in session now revalidates in the background.
Deactivated or expired sessions rejected immediately	Yes — deactivation is a row lookup on every admin request, and logout revokes durably.

Requirement	State
Direct URL access to a restricted module returns 403	Yes for the confined roles; the matrix says which.
Server validation errors shown clearly	Yes.
Exports show filter and date range	Yes.

---

14. Section 14 — Critical end-to-end scenarios

ID	Scenario	State
E2E-01	Guest → negotiation → booking → payment → confirmation → admin	✅
E2E-02	Cancellation → refund → admin ledger reconciliation	✅
E2E-03	Host onboarding → KYC → verification → publish → booking	✅
E2E-04	Host payout → finance approval → ledger → status	✅ **once** `FIELD_ENCRYPTION_KEY` **is set** — section 16
E2E-05	Offer → counter → accept → booking amount → commission	✅
E2E-06	Ticket → reply → status → closure → notification	✅
E2E-07	Compliance flag → review → resolution → audit	✅
E2E-08	Role change → existing session → permission changes	✅ role re-read per request
E2E-09	Admin deactivation → existing token rejected	✅
E2E-10	Payment/webhook replay → no duplicates	✅ row-locked
E2E-11	Concurrent admin actions → no double approval	✅
E2E-12	Deleted user/property → no orphan exposure	✅

Every one of these is exercised by the code paths and the test suite. What we have **not** done is drive them through the authenticated admin UI with real logins — section 17.

---

15. What we changed during this review

Three things were not true. All three are deployed.

Money was floating point

Sixteen columns were `DOUBLE(10,2)`: the booking total, the price, the tax, the refund, the pet fee, the host earning, the payout amount, the payout history, the booking-history price, three coupon limits and three property price fields.

The `(10,2)` rounds on **storage**, which is why nothing had visibly broken. It does not make the arithmetic exact — a DOUBLE is binary floating point, `0.1` has no exact representation in it, and `SUM()` over a column of them accumulates error. That is precisely the reconciliation drift section 6 asks about: booking total against payment against commission against host earning, differing by a few paise that nobody can account for, at the moment somebody is closing a month.

All sixteen are `DECIMAL(10,2)` now, applied to the live database. The migration reads the booking totals before and after and **refuses if they move**; they did not — ₹819,842.75 both times. Zero DOUBLE or FLOAT money columns remain. Nine model definitions still said `DOUBLE` and now agree with the schema.

Three tables had no indexes at all

`tbl_negotiation_offers` is the one that matters. Every read is by property and participants, and the concurrency guard takes `FOR UPDATE` on the thread — with only a primary key that lock is taken by **scanning the table**, so two guests negotiating on different properties contended with each other for no reason. It is 67 rows today and it is the table that grows fastest once negotiation is the product's main feature. Also `tbl_payout_reqs` (host, status) and `tbl_book_details` (booking, status).

A support account could change almost anything

29 of 191 admin endpoints carry an explicit role gate. `seo_manager` was already confined centrally against a named list of what it may reach; `support` was not — so a support login could approve a property, cancel a booking, delete a user, issue a coupon or rewrite the site's content. It could not touch finance, which is separately gated, and that is what the audit asked about specifically; this is the rest of least privilege.

Support is now confined the same way: write access to the four queues it owns, read access to the records behind a ticket, and nothing else. Reads that arrive as a POST are handled explicitly, because this API searches with POST and a GET-only rule would leave an agent able to open a record and unable to find one. A new endpoint is refused to the role until somebody chooses to add it — the failure mode worth having. Nobody is affected today: the one support account on the live database is inactive.

16. What needs you

1. `FIELD_ENCRYPTION_KEY` **(blocking host payouts).** Reported separately today with a screenshot: "Payout accounts can't be saved yet — the server is missing its encryption key." The server is right to refuse — it will not write a bank account number in plaintext — but the variable is not set on the host, and it was on neither the required nor the optional environment list, so nothing reported its absence. Measured: `tbl_host_acc_details` holds **zero rows**, so no host has ever saved an account and nothing is encrypted under a key that has since been lost. Generate one and set it:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

Never rotate it without re-encrypting the stored accounts. E2E-04 cannot complete until this is set.

2. `HEALTH_TOKEN`**.** Not set, which means the readiness *detail* at `/health/env` — which secrets are configured, whether the database variables match what the service is running on, whether payments can collect money — is readable by nobody, including you. The public answer still works. Set any long random string; it is the header `x-health-token`.

---

17. One thing we are not claiming

We do not enter stored account passwords into forms, so **the authenticated admin UI regression is yours to run rather than ours.** Everything reachable without signing in has been measured — the routes, the role gates, the database, the migrations, the public endpoints, the response headers — and everything else is verified from the code and the test suite, which is stated row by row above rather than blurred into a tick.

Concretely, what we cannot produce and you can: the API collection executed with real tokens for each of the five roles, and the admin UI evidence for the critical workflows. The attached matrix tells you exactly what to expect from each of the 191 endpoints for each role, which is the hard half of writing that collection.

---

18. Section 17 — Developer action required

Required update	Response
Backend version / commit	`2ba4345`, deployed
Admin frontend version / commit	`55c4f7d`, deployed
Database migration version	`20260906170000-operational-indexes` (four applied today)
P0 findings fixed	**6 of 6** — five were already implemented; the sixth, ADM-P0-01, was implemented and is now also evidenced by the attached matrix

Required update	Response
P1 findings fixed	**6 of 6** — five already implemented; ADM-P1-04 extended during this review
RBAC matrix attached	**Yes** — `AAJOO_ADMIN_RBAC_MATRIX_2026-09-06`, 191 endpoints × 5 roles, generated from the routes
API regression completed	Backend suite 63/63; every unauthenticated admin surface probed live
Database reconciliation completed	Money columns converted with a before/after total check that would have failed the migration; indexes added; soft-delete rule verified as a single definition
Payment live-mode verification	**Not possible without your live key.** The platform fails closed without one and announces its mode at boot
Security regression completed	Yes — sections 2, 3 and 12, including three defects found and fixed during this review and four found earlier today
Admin UI regression completed	**Not by us** — section 17
Guest / Host / Admin E2E completed	Code paths and suite yes; authenticated UI no
Known remaining issues	The two settings in section 16, and the date-storage note below

One thing called out rather than quietly changed. Stay dates in `tbl_book_details` are stored as `DD-MM-YYYY` **strings**. Every date comparison therefore runs `STR_TO_DATE(...)` per row, which no index can serve — so availability search cannot be indexed on dates however many indexes are added. It is correct today and it will not scale. Converting the column is a data migration with a real blast radius across booking, availability, calendar and reporting, and it belongs in its own change with its own testing rather than in an index migration. We would rather tell you it is there than fix it quietly and half-way.

---

19. Release position

🟢 **The admin control plane is ready for your authenticated regression.**

The audit's blocking position rested on six P0 findings. Five were already implemented before it was written — it says itself that it was reviewing a supplied archive — and this response gives the evidence for each rather than the assurance. The sixth is evidenced by the attached matrix. Three further defects were found during this review and fixed: money stored as floating point, three unindexed tables, and a support role that could change almost anything.

Two environment settings are yours, one of them blocking host payouts entirely.

Backend	`2ba4345` — 63/63 test files
Admin frontend	`55c4f7d`
Migrations applied to the live database	`guest-support-tickets`, `money-columns-decimal`, `money-columns-decimal-followup`, `operational-indexes`

Attached	`AAJOO_ADMIN_RBAC_MATRIX_2026-09-06`
Live verification	Finance route gates, admin route guards, `/health/push`, `/health/env`, `/db-test`, the socket handshake, and the money columns read back from the production database